

轻量开发专场

CANN开发者Meetup

2026.08.15 中国·南京



主办单位: CANN开源社区 协办单位:  AtomGit 开放原子紫金专区

CANN

Ascend C: 基于C API和Tensor API编程新体验

分享嘉宾：黄金华
2026.8.15 南京



 Ascend C 编程API新升级

 C API介绍以及编程实践

 Tensor API介绍以及编程实践

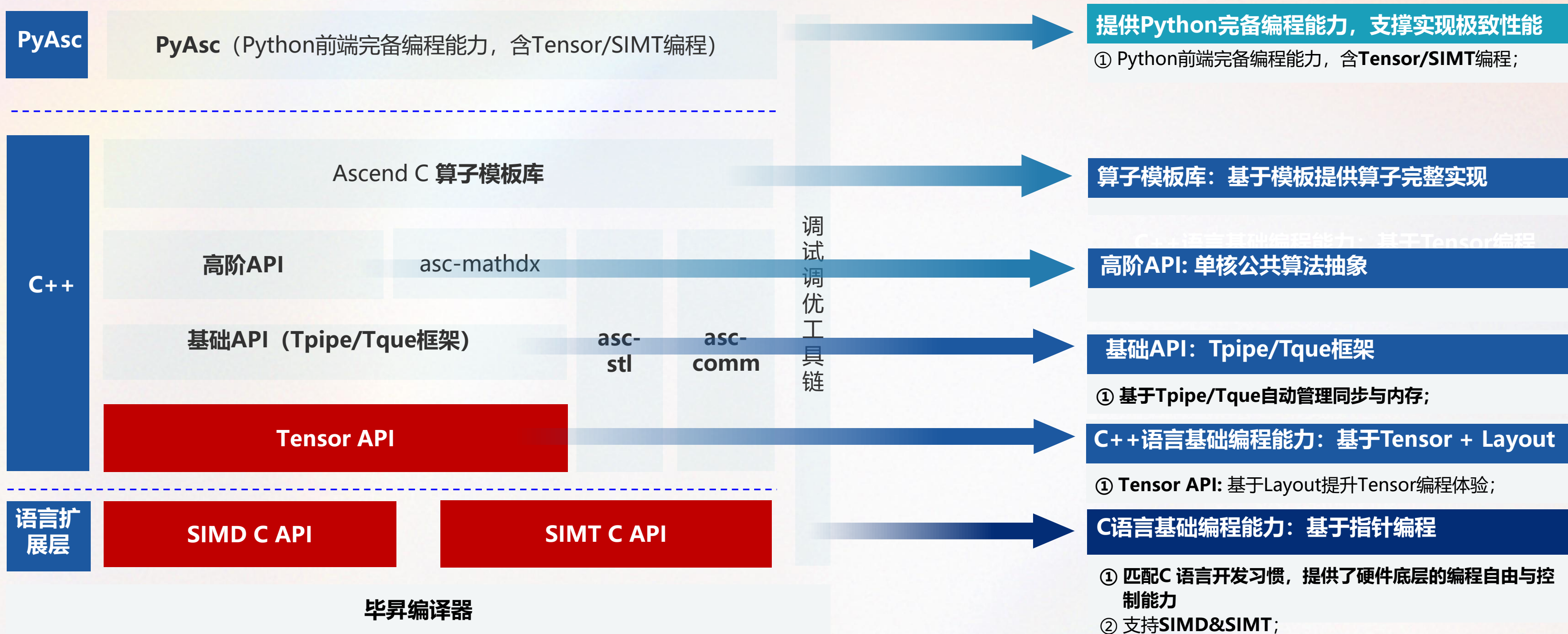
content

目录

Ascend C 编程API新升级

01

Ascend C 编程API全新升级



Ascend C 编程API全新升级

秉持「标准C/C++语法、最小化扩展」的核心设计原则，打造轻量化、高性能的底层编程基座

Tensor API

Tensor + Layout

02 | C++语言基础编程能力：基于Tensor + Layout

- **核心框架**：Tensor/Layout代数体系 + Arch/Atom/Algorithm API 层级；
- **核心特点**：张量与布局为一等公民、零成本编译期抽象、三层解耦，关注点分离、天然架构可移植性

语言扩展层 (C语言API)

SIMD C API + SIMT C API

01 | C语言基础编程能力：基于指针编程

- 统一命名风格，匹配C语言开发习惯；
- 原生指针编程；
- 指令级透明映射，零封装开销；
- 开放全部底层硬件能力，支撑精细化调优与极致性能实现

C API 介绍与编程实践

02

Ascend C 基于指针的C API编程概述

匹配C 语言开发习惯，提供了硬件底层的编程自由与控制能力

01 零开销原则 (Zero-overhead)

接口与硬件指令一一对应，无额外运行时封装与隐式调度，性能损耗完全透明、可量化预测，拒绝“黑盒”损耗。

02 原生指针编程 (Native Pointers)

支持开发者自主管理多级内存的分配、搬运与同步，打破高层抽象限制，实现对数据流向与算力调度的极致掌控。

03 命名风格统一 (Unified Style)

全系采用标准 snake_case 命名规范，深度贴合系统级 C 语言编程习惯，降低底层开发的认知负荷与代码迁移成本。

04 指令级透明映射 (Transparent Mapping)

单个 C API 直接映射单条硬件原语，无中间层转译损耗，确保每一行代码的执行效果精准可控，调试更高效。

 **极致算力场景：**大模型推理、科学计算等需榨干硬件算力的头部业务，突破性能天花板。

 **硬核开发群体：**面向精通底层架构，追求完全自主控制同步逻辑与内存布局的资深开发者。

Ascend C Memory 矢量算子(A2/A3)实践

匹配C 语言开发习惯，提供了硬件底层的编程自由与控制能力

```
__global__ __vector__ void add_custom(__gm__ float* x, __gm__ float* y,
__gm__ float* z)
{
    asc_init();
    ...
    // Allocate on-chip UB memory
    __ubuf__ float x_local[block_length];
    __ubuf__ float y_local[block_length];
    __ubuf__ float z_local[block_length];

    // Copy input data from Global Memory (i.e., Device Memory) to on-chip
    UB memory
    asc_copy_gm2ub(x_local, x_gm, block_length * sizeof(float));
    asc_copy_gm2ub(y_local, y_gm, block_length * sizeof(float));
    asc_sync();

    // Call SIMD API to complete the Add operation
    asc_add(z_local, x_local, y_local, block_length);
    asc_sync();

    // Write the result from on-chip UB memory back to Global Memory
    asc_copy_ub2gm(z_gm, z_local, block_length * sizeof(float));
    asc_sync();
}
```

① 核函数定义

② 内存申请

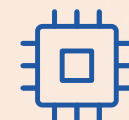
③ 同步管理

④ 计算/搬运类接口等



极简语法，无缝迁移

沿用标准 C 语言逻辑结构，仅需掌握少量硬件专属关键字（如 __gm__ / __ubuf__），熟悉系统编程的开发者即可快速上手，学习成本极低。



显式控制，极致效能

通过显式内存层级管理与向量化指令调度，开发者可直接操控硬件特性，消除隐式开销，充分挖掘昇腾 AI 芯片的计算潜能。

Ascend C Reg矢量编程算子(950)实践

匹配C 语言开发习惯，提供了接近硬件底层的终极编程自由与控制能力

```
__simd_vf__ inline void add_vf(  
    __ubuf__ float* x_local, __ubuf__ float* y_local,  
    __ubuf__ float* z_local, uint32_t data_len)  
{  
    constexpr uint16_t num_elems = asc_get_vf_len() / sizeof(float);  
    uint16_t repeat_time = (data_len + num_elems - 1) / num_elems;  
  
    vector_bool mask_reg;  
    vector_float src0_reg;  
    vector_float src1_reg;  
    vector_float reg_dst;  
    for (uint16_t i = 0; i < repeat_time; ++i) {  
        mask_reg = asc_update_mask_b32(data_len);  
        asc_loadalign(src0_reg, x_local + i * num_elems);  
        asc_loadalign(src1_reg, y_local + i * num_elems);  
        asc_add(reg_dst, src0_reg, src1_reg, mask_reg);  
        asc_storealign(z_local + i * num_elems, reg_dst);  
    }  
}
```

① VF函数定义

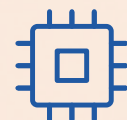
② 寄存器定义

③ 计算/搬运类接口等



极简语法，无缝迁移

沿用标准 C 语言逻辑结构，仅需掌握少量硬件专属关键字（如 __gm__ / __ubuf__），熟悉系统编程的开发者即可快速上手，学习成本极低。



显式控制，极致效能

通过显式内存层级管理与向量化指令调度，开发者可直接操控硬件特性，消除隐式开销，充分挖掘昇腾 AI 芯片的计算潜能。

Tensor API 介绍与编程实践

03

Tensor API 概述

基于C API构建，引入布局代数体系，将多维 Tensor 及其内存 Layout 提升为编程一等公民

Arch层
基于C API，指令封装层

Atom 层
原子语义与元信息层

Algorithm层
通用算法接口

Tensor + Layout （以及Layout的代数运算）

Ascend C语言扩展层C API

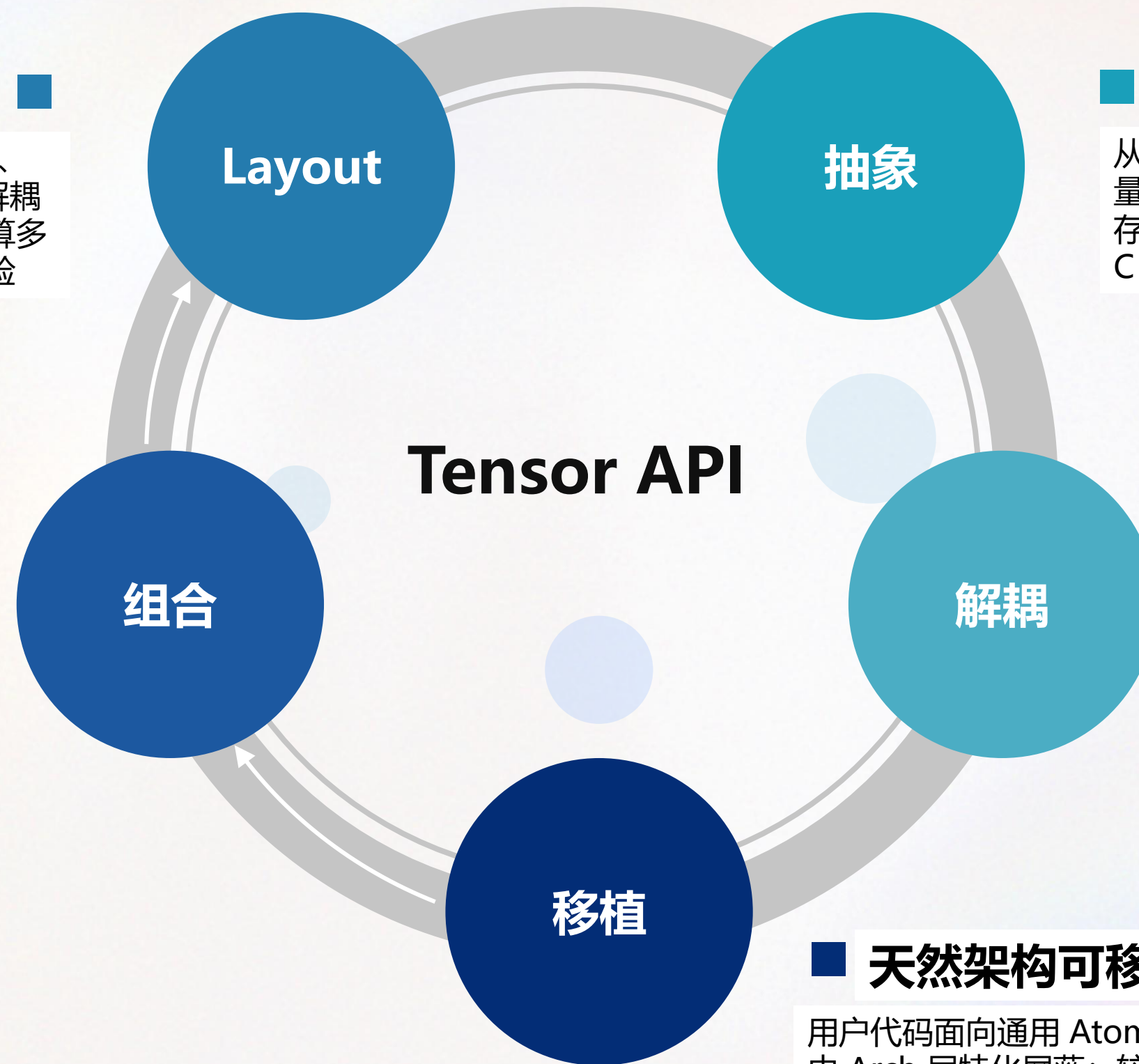
Tensor API的特点

张量与布局为一等公民

基于布局代数体系，将多维 Tensor、内存 Layout 作为核心编程对象；解耦逻辑视图与物理存储，无需手工计算多维数组地址偏移，大幅提升编程体验

乐高式高度可组合

任意 Atom 可适配任意 Algorithm，任意 Layout 可结合任意 Atom；通过模块化积木式拼接构建内核，大幅降低高性能 Kernel 开发成本



零成本编译期抽象

从 Arch 到 Algorithm 全层均为轻量编译期封装；无虚函数、无动态内存分配，运行时指令性能与手写原生 C 代码完全等价

三层解耦，关注点分离

硬件指令（Arch）、原子层（Atom）、算法层（Algorithm）独立演化；新架构仅需新增 Arch 层特化，Atom 与 Algorithm 上层代码几乎无需修改

天然架构可移植性

用户代码面向通用 Atom / Algorithm 接口编程，硬件差异由 Arch 层特化屏蔽；较好支持跨架构、跨芯片的代码迁移与复用。

Tensor API 示例 (1): Tensor与Layout定义

基于NZ、ZN格式定义Tensor

```
using namespace asc::te;

// 分配内存
__cbuf__ half l1_a_buf[base_m * base_k];
__cbuf__ half l1_b_buf[base_k * base_n];
__ca__ half l0_a_buf[base_m * base_k];
__cb__ half l0_b_buf[base_k * base_n];

// 基于Layout定义Tensor
auto l1_a_tensor = make_tensor(
    make_mem_ptr(l1_a_buf),
    make_frame_layout<NZLayoutPtn, half>(base_m,
base_k));

auto l1_b_tensor = make_tensor(
    make_mem_ptr(l1_b_buf),
    make_frame_layout<ZNLayoutPtn, half>(base_k,
base_n));
...
```

定义Tensor, 并基于slice接口切分子Tensor

```
using namespace asc::te;

// 定义 copy atom

auto tensor_a_gm = make_tensor(
    make_mem_ptr(x),
    make_frame_layout<NDLayoutPtn>(M, K));

auto tensor_b_gm = make_tensor(
    make_mem_ptr(y),
    make_frame_layout<DNLayoutPtn>(K, N));

auto block_a_gm =
    tensor_a_gm.slice(
        make_coord(m_core_idx * block_m, 0),
        make_shape(block_m, block_k));

...
```

张量与布局为一等公民

Tensor API 示例 (2): 数据搬运与矩阵计算

基于Algorithm接口和Tensor完成数据搬运

矩阵计算

基于Algorithm接口和Tensor完成矩阵计算

```
using namespace asc::te;
```

```
// 定义 copy atom
```

```
auto copy_gm2l1_atom = make_copy(  
    CopyGM2L1{}, CopyGM2L1TraitDefault{});
```

```
// 调用 copy algorithm 接口
```

```
copy(copy_gm2l1_atom,  
    l1_a_tensor,  
    gm_a_single.slice(  
        make_coord(mi * base_m, ki * base_k),  
        make_shape(base_m, base_k)));
```

```
using namespace asc::te;
```

```
// 定义 mmad atom
```

```
auto mmad_atom =  
    make_mmad(MmadOperation{},  
        MmadTraitDefault{});
```

```
// 调用Mmad Alogrithm 接口
```

```
mmad_params params{base_m, base_n, base_k, 0,  
    (ki == 0)};
```

```
mmad(mmad_atom.with(params),  
    c_tensor, a_tensor, b_tensor);
```

零成本编译期抽象、Arch/Atom/Algorithm三层解耦、乐高式高度可组合、天然架构可移植性

基于Tensor API的Matmul(950)示例

```
constexpr uint32_t mutex_id = 0;

for (uint32_t mi = 0; mi < m_loop; ++mi) {
    for (uint32_t ni = 0; ni < n_loop; ++ni) {
        for (uint32_t ki = 0; ki < k_loop; ++ki) {

            asc_lock(PIPE_MTE2, mutex_id);
            copy(copy_gm2l1_atom, l1_a_tensor, gm_a_single.slice(make_coord(mi * base_m, ki * base_k), make_shape(base_m,
base_k)));
            copy(copy_gm2l1_atom, l1_b_tensor, gm_b_single.slice(make_coord(ki * base_k, ni * base_n), make_shape(base_k,
base_n)));
            asc_unlock(PIPE_MTE2, mutex_id);

            asc_lock(PIPE_MTE1, mutex_id);
            copy(copy_l12l0a_atom, l0_a_tensor, l1_a_tensor);
            copy(copy_l12l0b_atom, l0_b_tensor, l1_b_tensor);
            asc_unlock(PIPE_MTE1, mutex_id);

            asc_lock(PIPE_M, mutex_id);
            mmd_params params{base_m, base_n, base_k, 0, (ki == 0)};
            mmd(mmd_atom.with(params), l0_c_tensor, l0_a_tensor, l0_b_tensor);
            asc_unlock(PIPE_M, mutex_id);
        }

        asc_lock(PIPE_FIX, mutex_id);
        copy(copy_l0c2gm_atom, gm_c_single.slice(make_coord(mi * base_m, ni * base_n), make_shape(base_m, base_n)),
l0_c_tensor);
        asc_unlock(PIPE_FIX, mutex_id);
    }
}
```

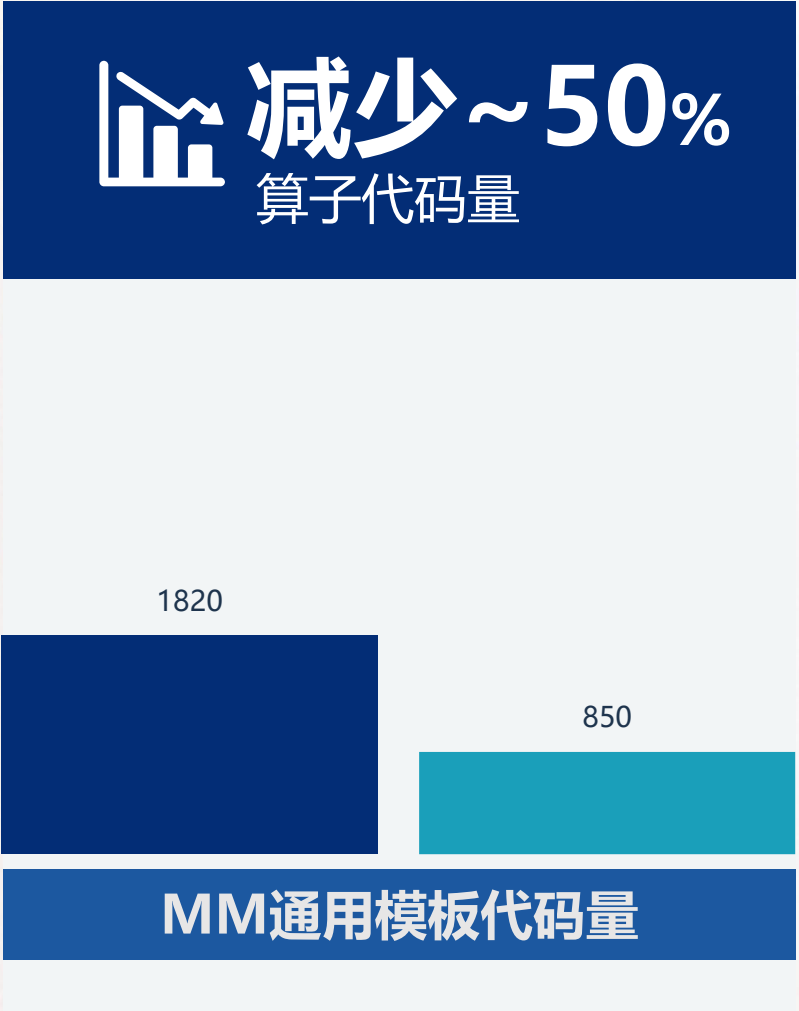
较容易实现Tensor数据搬运

Ascend 950新增特性：基于Mutex ID实现对PIPE_MTE2、PIPE_MTE1、PIPE_M同步

基于Tensor的矩阵计算

较容易实现分块数据搬运

CANN 商用算子Matmul(950)通用模板实践



基线版本 (手工计算)



基于Tensor API实现

学习推荐

- Ascend C API 实现与样例: <https://gitcode.com/cann/asc-devkit>
- Ascend C 编程指南: <https://asc.gitcode.com>
- CANN 算子领域样例仓库: <https://gitcode.com/cann/cann-samples/tree/master/Samples/>
- CANN Learning Hub: <https://gitcode.com/cann/cann-learning-hub/>

THANKS



参与互动答疑贴赢取礼品



PyPTO: 基于Tensor/Tile的算子编程体验

分享嘉宾: 高翔
2026.8.15 中国·南京



CANN

content

目录

- 01** PyPTO整体介绍
- 02** PyPTO-Pro的设计初衷
- 03** PyPTO-Pro的易用性来源
- 04** PyPTO-Pro尝鲜体验

PyPTO整体介绍

01

PyPTO：昇腾原生亲和 Python Tensor/Tile编程

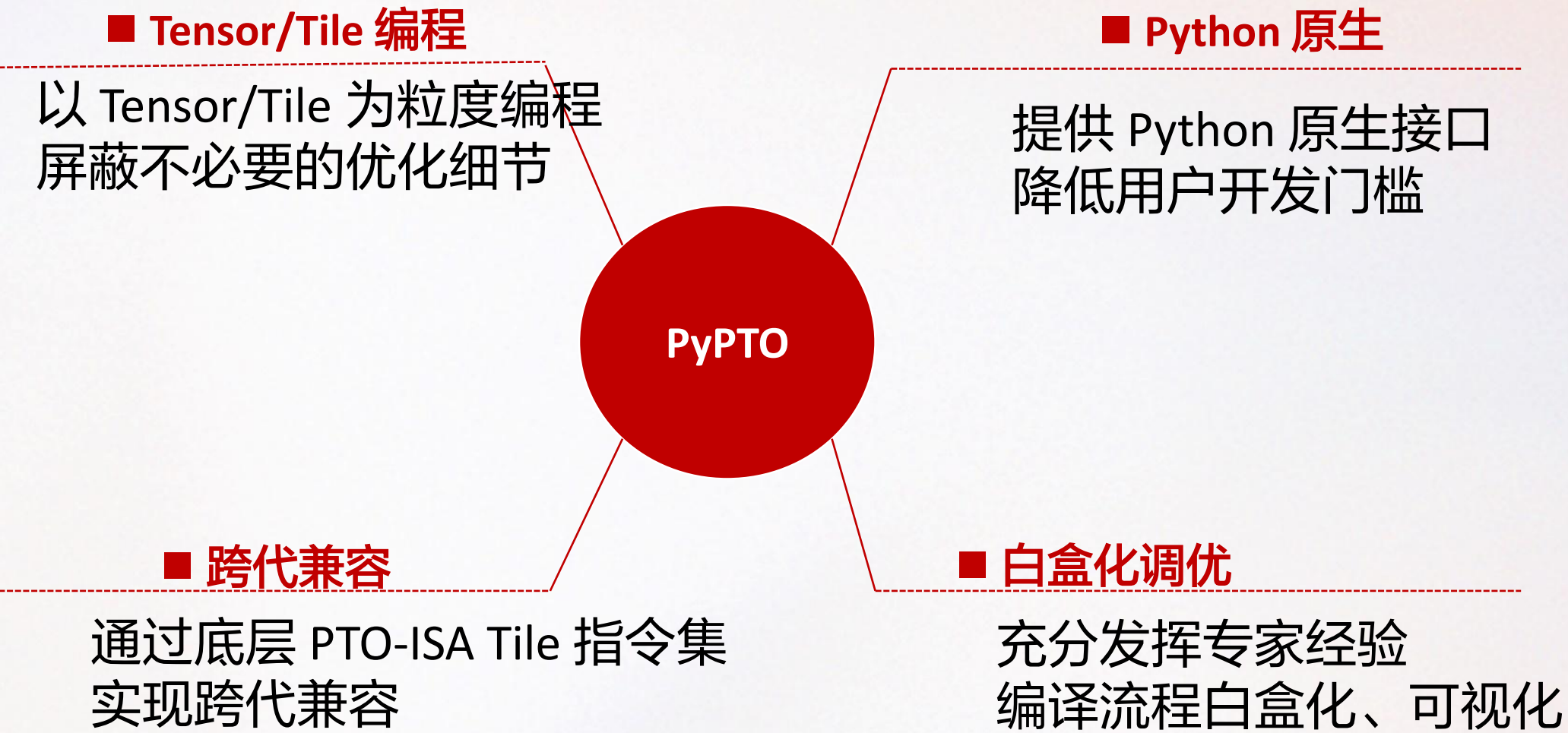
PyPTO 是什么？

- **本质：** 面向 AI 加速器的高性能编程框架
- **目的：** 简化算子开发流程，同时保持高性能计算能力
- **范式：** 创新的 PTO 编程
- **理念：** 基于 Tensor/Tile 的编程模型

PyPTO 的定位

- 不仅限于单个算子的开发，更强调**复杂融合算子，甚至于整个模型网络**的开发能力
- 以**Tile块为最基础的数据结构抽象**，以此来简化算子编程

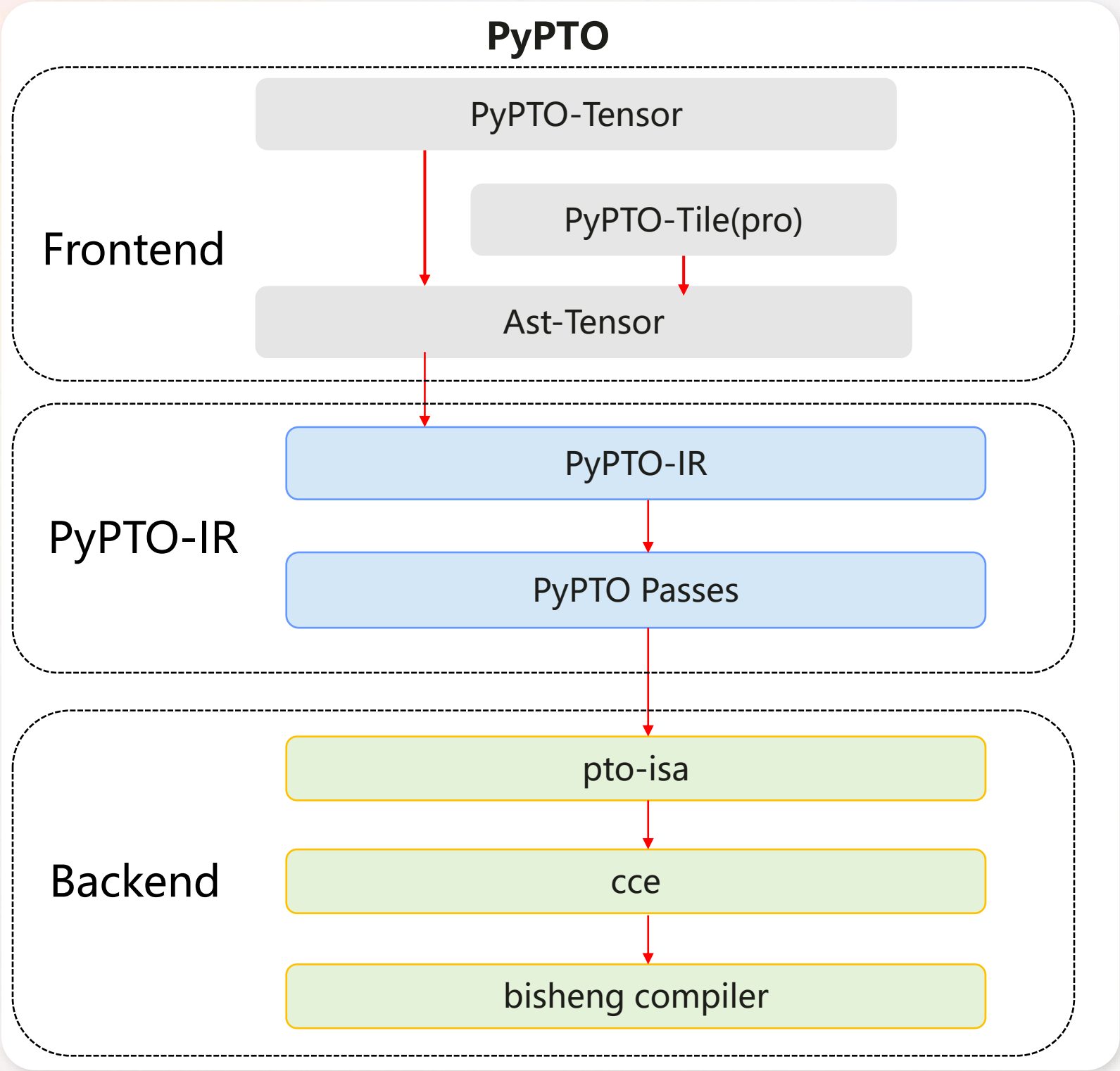
<https://gitcode.com/cann/pypto>



PyPTO：昇腾原生 Python Tensor/Tile编程架构

算子编程语言需要描述算子的**计算逻辑**、**调度策略**以及**指令映射**，而调度策略和指令映射是需要针对硬件做特殊设计的，实现难度较大。

- PyPTO-Tensor：创新性提出Tile切分+自动MPMD调度方案，前端只需要描述计算逻辑，**调度和指令映射交给框架**。易用性最好，但是由于调度开销和切分算法的局限性，性能做不到极致。
- PyPTO-Tile (pro)：对标**CuTeDSL**，暴露的buffer控制，通过一系列措施，**简化了调度策略和指令映射**的写作。补充了Tensor编程的性能短板。



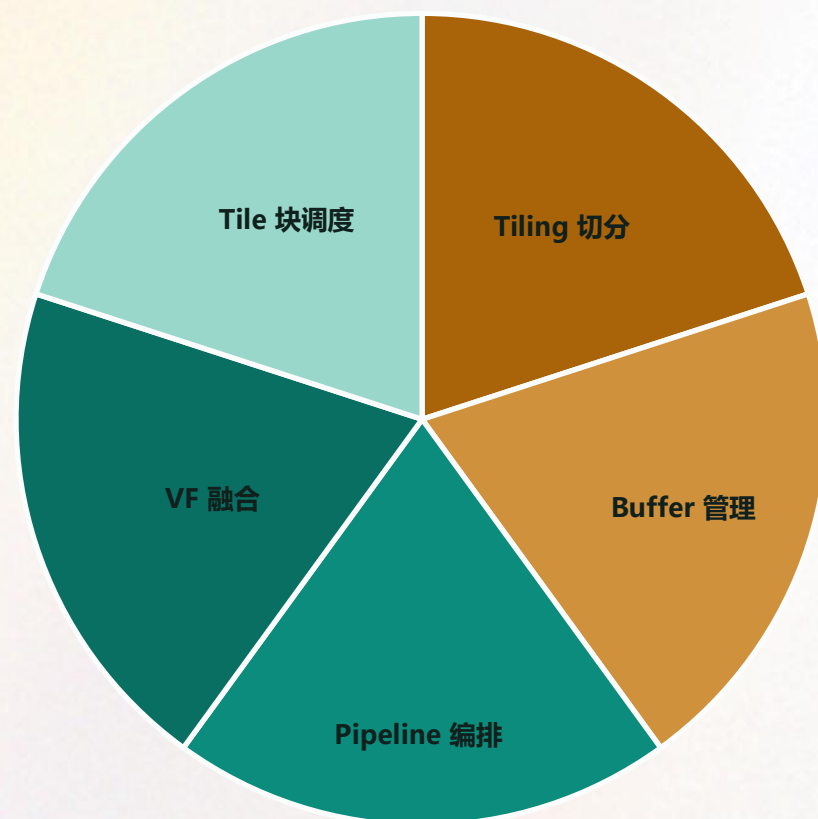
维度	PyPTO-Tensor	PyPTO-Tile(pro)
IR表达	PyPTO-IR	PyPTO-IR
前端抽象	Tensor-Level	Tile-Level
内存层级	不感知	感知
内存分配	不感知	手动分配
内存复用	不感知	手动复用
内存抽象	不感知	Tile + BufferManager
核内同步	不感知	不感知
CV自动分离	不感知	不感知
核间同步	不感知	不感知
Tile表达	多维	2维
Offset计算	传入坐标自动计算	传入坐标自动计算
Tensor表达	Shape + Stride+ Layout	Shape + Stride + Layout
VF融合	利用bisheng能力	利用bisheng能力
VF手写能力	支持	支持
Tiling表达	单tiling key	多tilingkey 二进制
性能	0.6+理论性能	1.0 理论性能

PyPTO-Pro的设计初衷

02

Ascend 950算子编程5大维度

从“数据怎么分”到“一条指令怎么填”，一个高性能算子的设计空间可以拆成五个维度。



- 1 Tiling 切分**
数据如何均衡的分给多个核；
同时每个核的计算和搬运效率需要最高
- 2 Buffer 管理**
Buffer如何复用；多块 buffer 怎么轮转，轮转之后同步如何插入
- 3 Pipeline 编排**
Cube和Vector间并行度最大化，同时插入对应的核间同步
- 4 VF 融合**
手写VF融合性能好，但是难写
- 5 Tile 块调度**
基本块的搬运、计算映射成硬件指令

多核切分、负载均衡

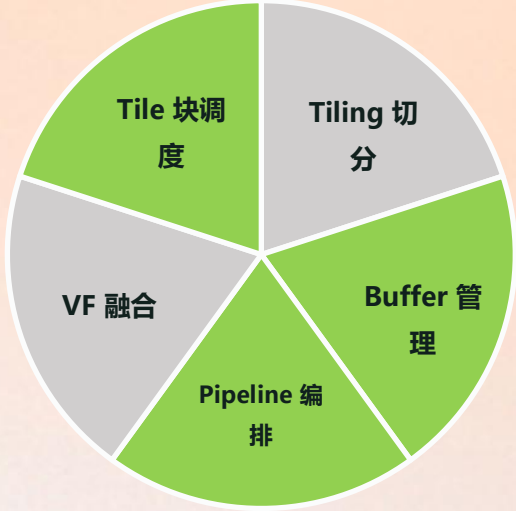
核内复用+轮转+同步

CV核间流水 & 同步

Vector性能提升

硬件参数填写

PyPTO-Pro致力于通过合理的取舍，保留易用性的同时获取高性能



基于多年算子开发经验，将不影响性能的指令和写法充分封装，最大化提升易用性；
判据只有一条：**这个维度在不同算子之间差异大吗？** 差异大的是设计，必须由懂这个算子的人来做；差异小的是套路，套路就该被封装掉。



虚线框 = 已封装，但保留下沉通道

交给开发者 · 2 件

Tiling 切分 · Buffer 复用

直接决定性能上限，而且每个算子都不一样——没有通解，只能由懂它的人来做。

封装掉 · 3 件

轮转同步 · Pipeline · Tile 调度

都有唯一正确答案，编译器推得出来。后面会详细介绍

封装 + 可下沉 · 1 件

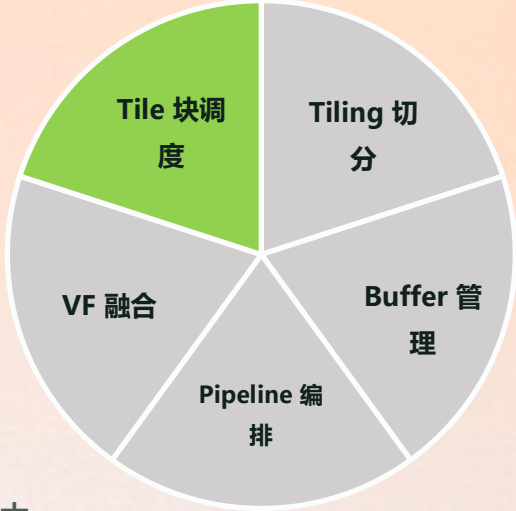
VF 融合

Tile 级接口让写法大幅简化；代价是中间结果走 UB，比手写 vf.* 略慢。

PyPTO-Pro的易用性来源

03

Tile粒度调度：所有操作均以Tile为核心



Tile 是一个**硬件感知的二维数据块**。你用 tile 坐标描述它在 Tensor 上的位置，load_tile / store_tile 自动把坐标乘上 tile shape，指令参数一并封装。

传统写法：先做一遍算术

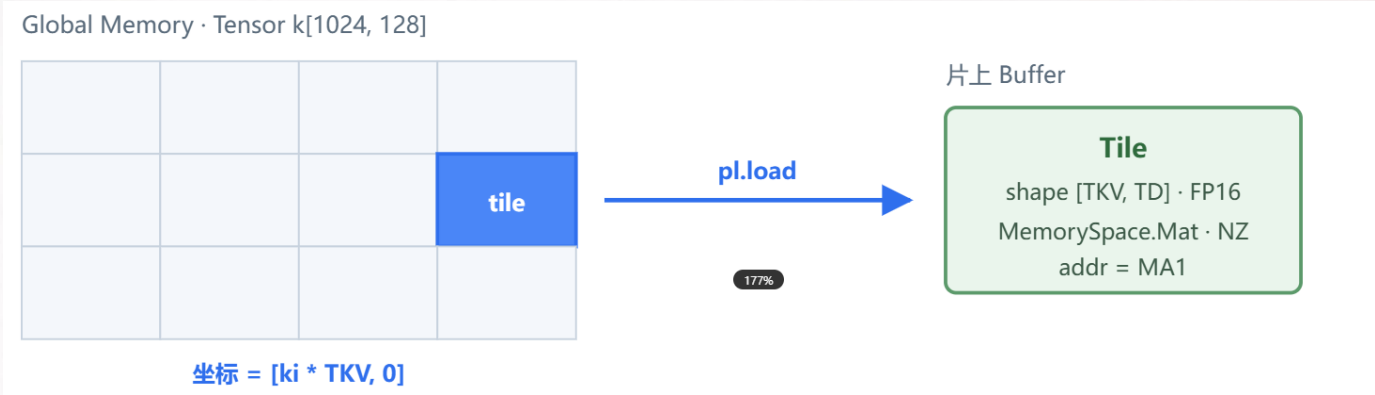
```
# 手工计算offset
Off_q = ((b * n + h) * seq)
Off_k = ki * TKV
Off_all = (off_q + off_k) * head_dim

# 再填指令参数burst / brusr_len/ 源步长 / 目的步长 ...
Copy_gm_to_cbuf(dst, src + off_all, nB, lenB, sStride, dStride)
```

PyPTO Pro：直接说第几块

```
pl.load_tile(tile_a, x, [i, j])    # 第(i,j) 块
pl.store_tile(z, tile_c, [i, j])

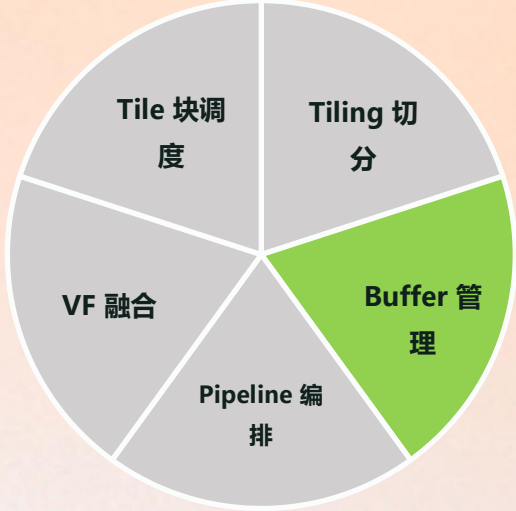
# 需要绝对元素偏移时，也有：
pl.load(tile_a, x, [row, col])
pl.add(tile_c, tile_a, tile_b)
pl.store(z, tile_c, [row, col])
```



- **搬运按 tile 索引**：偏移 = 索引 × tile shape，由框架算。
- **指令参数被 Tile API 封装**：burst、步长、格式这些不再出现在你的代码里。
- **仍然具备足够的硬件控制力**：通过stride、layout、地址的设置依然可以完成复杂的转置、复用、跳读跳写等操作

- 这一条是所有优化的基础：
- Tile是调度的核心单元，除了简化了指令参数填写；
 - Tile还为核内以及核间同步的自动插入提供了载体，是后续两个优化点的基础

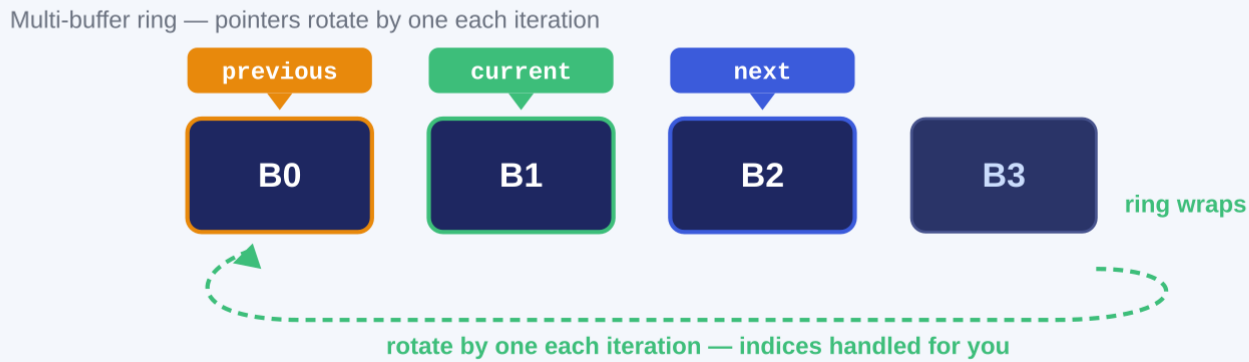
Buffer轮询 & 核内同步过程



1 You write — request a tile block by role



2 Framework handles rotation + synchronization



- ✓

Buffer rotation
handled automatically — no manual buffer indices
- ✓

Intra-core synchronization
`set_flag / wait_flag` inserted for you

痛点:

- 用户需要手动指定多Buffer轮询的Index
- 用户需要插入核内同步，容易引起避免死锁和漏插同步

Manual synchronization



⚠ hand-placed event IDs — easy to deadlock or race

pypto-pro: auto-inserted



Dependencies inferred from data flow. The compiler places every barrier correctly — **no manual synchronization to write.**

TileGroup + auto_mutex: 核内同步自动补全

框架在每次使用轮转 tile 的前后自动发出 lock / unlock。

前端代码

```
@pl.jit(auto_mutex=True)

# 双缓冲(Double Buffer): depth就是深度
a_db = pl.make_tile_group(
    type=tt, addr=0x0000, depth=2)

# 循环里每轮取下一块
tile_a = a_db.next()
pl.load_tile(tile_a, x, [i, j])
pl.add(tile_c, tile_a, tile_b)
```

- 省去了核内正反向同步的插入
- 想从双缓冲改成三缓冲？把 depth改成3，其余不动。

通过950芯片的mutex_id机制来对tile进行加锁保护
相比于set_flag/wait_flag更灵活

编译器补上的（青色）

```
tile_a = a_db.next()
tile_a.cursor = (tile_a.cursor + 1) % 2
cursor = tile_a.cursor
| mutex_lock(pipe=MTE2, id=cursor) 自动插入
pl.load_tile(tile_a, x, [i, j])
| mutex_unlock(pipe=MTE2, id=cursor)
| mutex_lock(pipe=V, id=cursor)
pl.add(tile_c, tile_a, tile_b)
| mutex_unlock(pipe=V, id=cursor)
```

TileGroup: 声明「哪几块 Buffer 在轮转」，同步交给框架

改流水级数 = 改一个列表长度，不是重写同步代码

slot 0
mutex_id = 10
addr = CA0

slot 1
mutex_id = 11
addr = CA1

slot 2
mutex_id = 12
addr = CA2

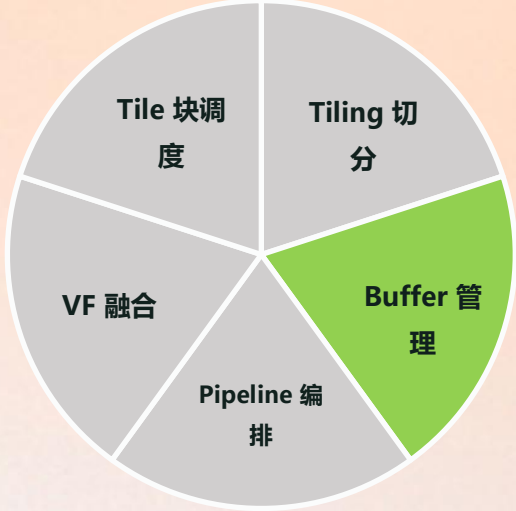
slot 3
mutex_id = 13
addr = CA3

next() 环形轮转

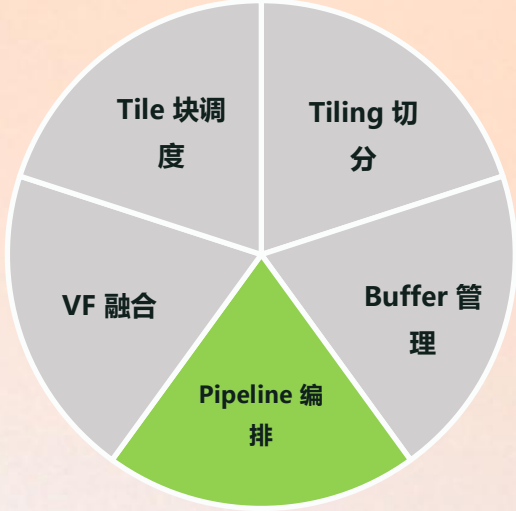
```
acc_db = pl.make_tile_group(type=tt, addr=CA0, mutex_ids=[10, 11, 12, 13])
cur = acc_db.next() ← 你只写这一行
```

auto_mutex=True 时框架自动展开为:

```
mutex_lock(M, 10) → pl.matmul(cur, ...) → mutex_unlock(M, 10)
```



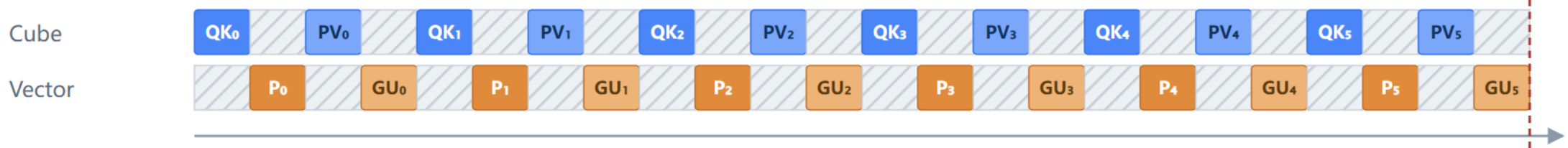
核间同步与流水编排的过程



核间流水的完整过程：填充 → 稳态 → 排空

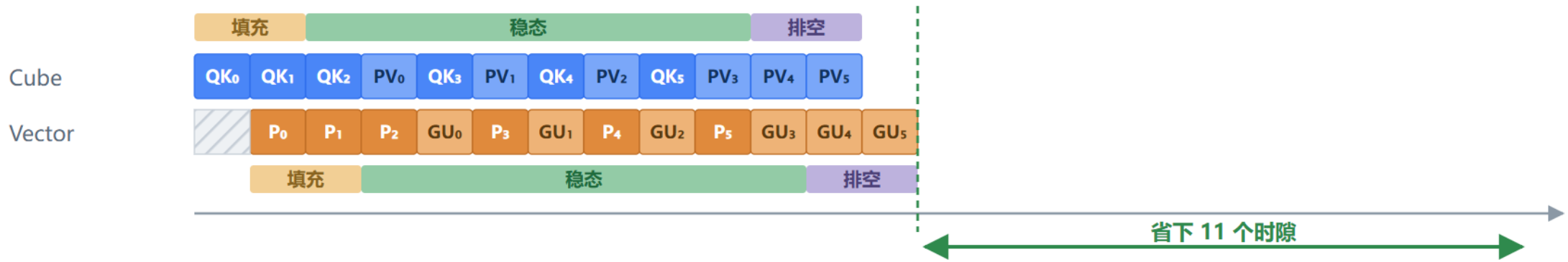
FlashAttention, N=6 个 KV 分块; preload=2 推导出 stage 延迟 $QK=0 \cdot P=1 \cdot PV=2 \cdot GU=3$, ring depth=4

✗ 串行执行



每块 KV 的 4 个 stage 首尾相接：一个核在算，另一个核必然在等

✓ preload=2 自动流水



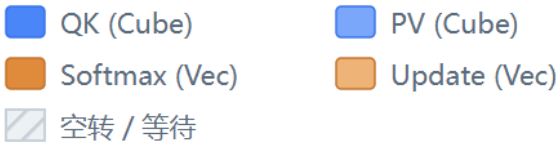
填充：Cube 连续跑 QK₀ QK₁ QK₂ —— 同色背靠背，因为下游 stage 被 ctx.is_valid 挡住，不发指令也不停顿

稳态：两条泳道都在换色 —— Cube 在 QK/PV 间交替，Vector 在 P/GU 间交替，双核同时满载

排空：Cube 连续跑完 PV₄ PV₅ 收工，Vector 再连续跑 GU₄ GU₅ 收尾

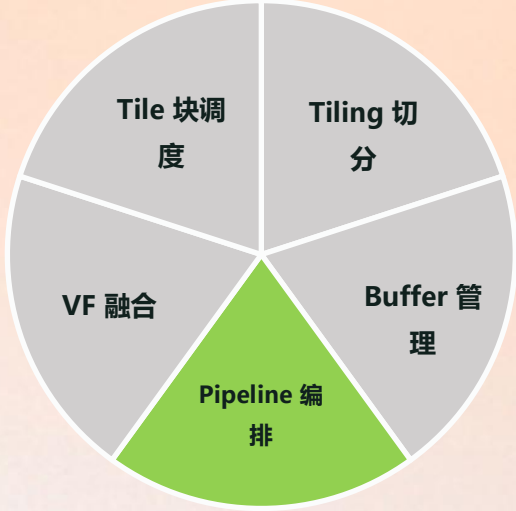
全程唯一的真实等待，是 Vector 开头等 QK₀ 的那 1 个时隙；此后无气泡

N=6: 24 → 13 时隙 (1.85×)，核利用率 50% → 92%；N 越大，填充/排空占比越小，越接近理论上限 2×



手写这套流水意味着：

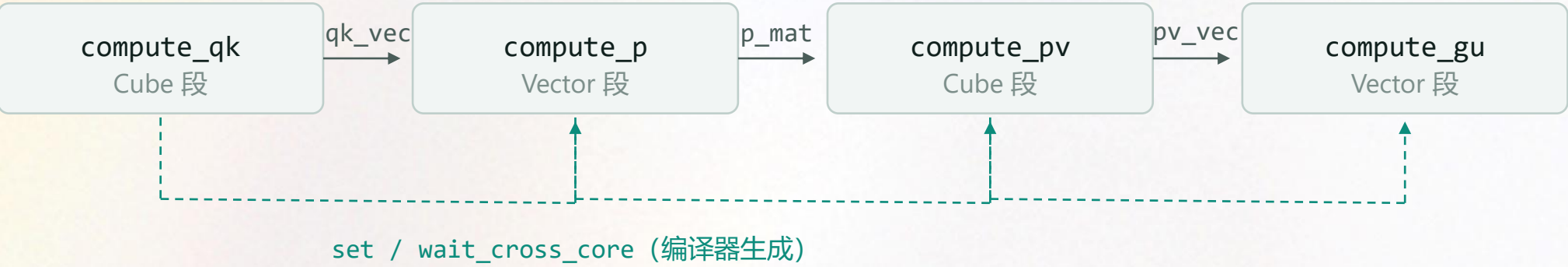
- 循环要按流水深度展开
- 每个阶段的输入要从不同深度历史迭代里取
- 核间同步的EventID 要按 ring buffer 索引正确收发，且保证配对
- 改流水深度需要重新适配代码



@pl.pipeline.stage: 核间同步与流水编排交给编译器

用 @pl.pipeline.stage 把每个计算阶段标出来，编译器**分析各阶段对 tile 的读写依赖**，自动插入 Cube 与 Vector 之间的核间同步，并把串行阶段变换成 并行流水。

你写的：4 个 @pl.pipeline.stage



—— 实线：stage 之间共享 tile 形成的数据依赖 --- 虚线：编译器据此生成的跨核同步与流水重叠
全程没有一行 set_cross_core 。

开发者只需要写串行的流程

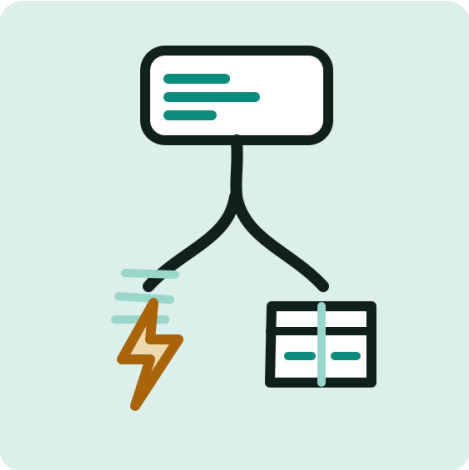
```
@pl.pipeline.stage
# Cube 阶段: 写 qk_vec
def compute_qk(...): ...

@pl.pipeline.stage
# Vector 阶段: 读 qk_vec, 写 p_mat
def compute_p(...): ...

@pl.jit(auto_mutex=True,
        pipeline=pl.pipeline.PipelineConfig(
            preload=2))
def fa_kernel(...):
    for ki in pl.range(0, skv_tiles):
        with pl.section_cube():
            compute_qk(...)
```

- 调深调浅只改一个数：preload=2 → 3，预热 / 稳态 / 收尾由编译器重新展开。

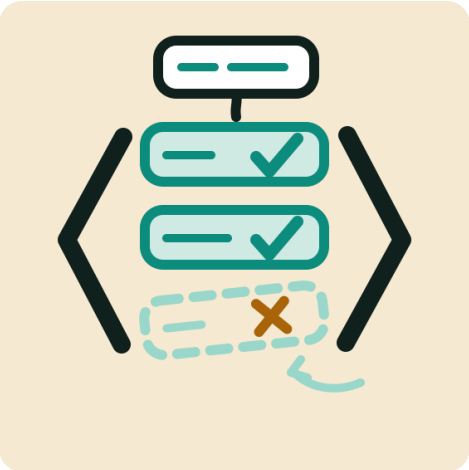
把 “能写” 变成 “好写”



01
一份代码，JIT 调完直接发二进制

同一份 kernel 源码，不用为了发布再改写一遍。JIT 下快速迭代，调完配合 aclnn + tiling 直接二进制发布。

@pl.jit → aclnn + tiling



02
有效 tiling_key 用 Python 描述

哪些 tiling_key 组合有效，直接用 Python 写出来，无效的组合不必生成。不用再手工维护一张分支表。

tiling_key



03
Python前端print / dump_tensor 全支持

想看哪一步就打哪一步，tensor 直接 dump 出来比对。

print / dump_tensor



04
先跑通功能，再手写 VF 压性能

完整的 tile 级算子库让功能先跑起来；性能不够时，热点上换成手写 vf.*。抽象是台阶，不是天花板。

tile 算子库 → vf.*

PyPTO-Pro尝鲜体验

04

先导版本已经在pypto仓开源发布

安装8月最新社区包即可体验：

- 代码仓路径：<https://gitcode.com/cann/pypto>
- 快速上手：https://gitcode.com/cann/pypto/tree/master/docs/zh/pypto_pro/tutorials/quick_start
- API文档：https://gitcode.com/cann/pypto/tree/master/docs/zh/pypto_pro/api
- Jit算子样例路径：https://gitcode.com/cann/pypto-gym/tree/master/src/pypto_gym/ops/pypto_pro
- 二进制发布的算子样例路径：
 - https://gitcode.com/cann/ops-transformer/tree/master/attention/dense_lightning_indexer_softmax_lse_v2
 - https://gitcode.com/cann/ops-transformer/tree/master/attention/sparse_flash_mla_softmax_l1_norm

THANKS



参与互动答疑贴赢取礼品



轻量开发专场

CANN开发者Meetup

2026.08.15 中国·南京



主办单位: CANN开源社区 协办单位:  AtomGit 开放原子紫金专区

CANN

Mooncake x HIXL: KV Pool技术分享

HIXL单边通信库提供高效点对点传输能力，构建KV Pool传输基座，加速大模型推理

分享嘉宾：游霄
2026.8.15 中国·南京



 KV Pool使用场景

 KV Pool底层传输：HIXL单边通信库

 KV Pool部署示例

 KV Pool演进方向

大模型推理中的KV复用



PREFILL

整段 prompt 并行计算

矩阵乘占比高 · 算术强度高

主要约束：算力 / 批处理效率

看 TTFT

DECODE

一次生成一个新 token

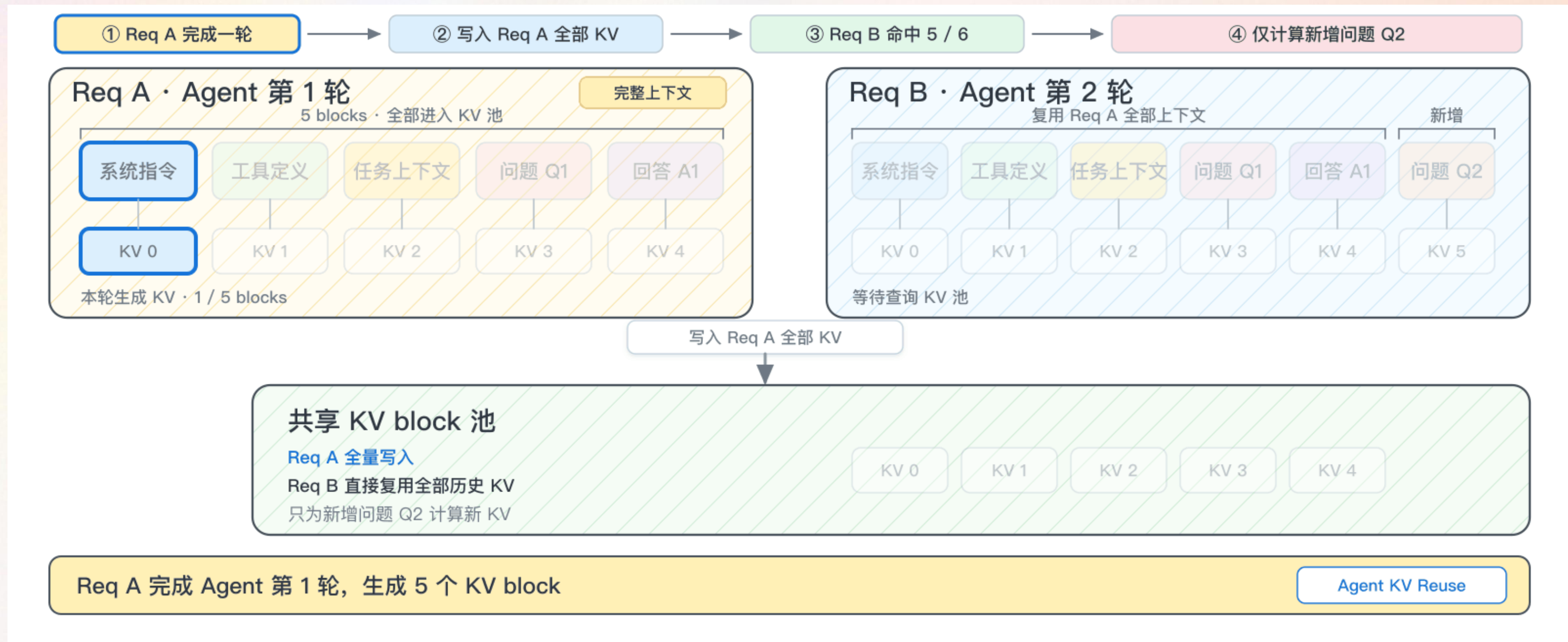
反复读取历史 KV · 小步迭代

主要约束：Device内存带宽 / 通信

看 TPOT

大模型推理的两阶段：Prefill计算用户的完整请求产生首Token和KV, Decode复用前面的KV产生后续的Token

相同前缀的 KV，跨请求能不能复用？



相同前缀的KV直接复用，大幅减少TTFT耗时

跨请求复用问题: KV Cache 占用空间大

$$\text{KV bytes} \approx 2 \times \text{layers} \times \text{kv_heads} \times \text{head_dim} \times \text{dtype_bytes} \times \text{seq_len}$$

2

同时保存 K 与 V

结构

 $\text{layers} \cdot \text{kv_heads} \cdot \text{head_dim}$

精度

FP16 / BF16 常见为 2 bytes

长度

 seq_len 每翻倍, KV 也翻倍 $\approx 320 \text{ KB}$

每 token

×

× 128K

长上下文

=

 $\approx 40 \text{ GB}$

每请求 KV

×

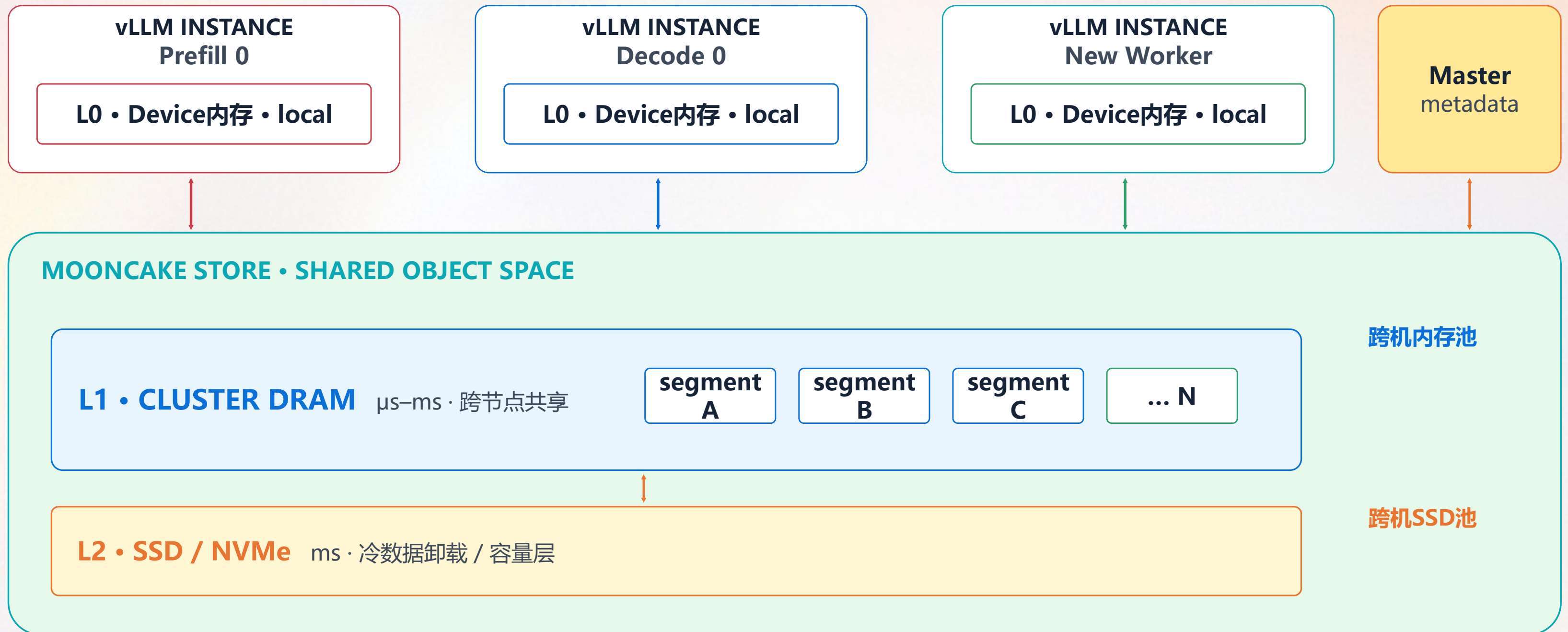
× 并发

实例总需求

1K个用户请求: 占用~40TB, 单机放不下

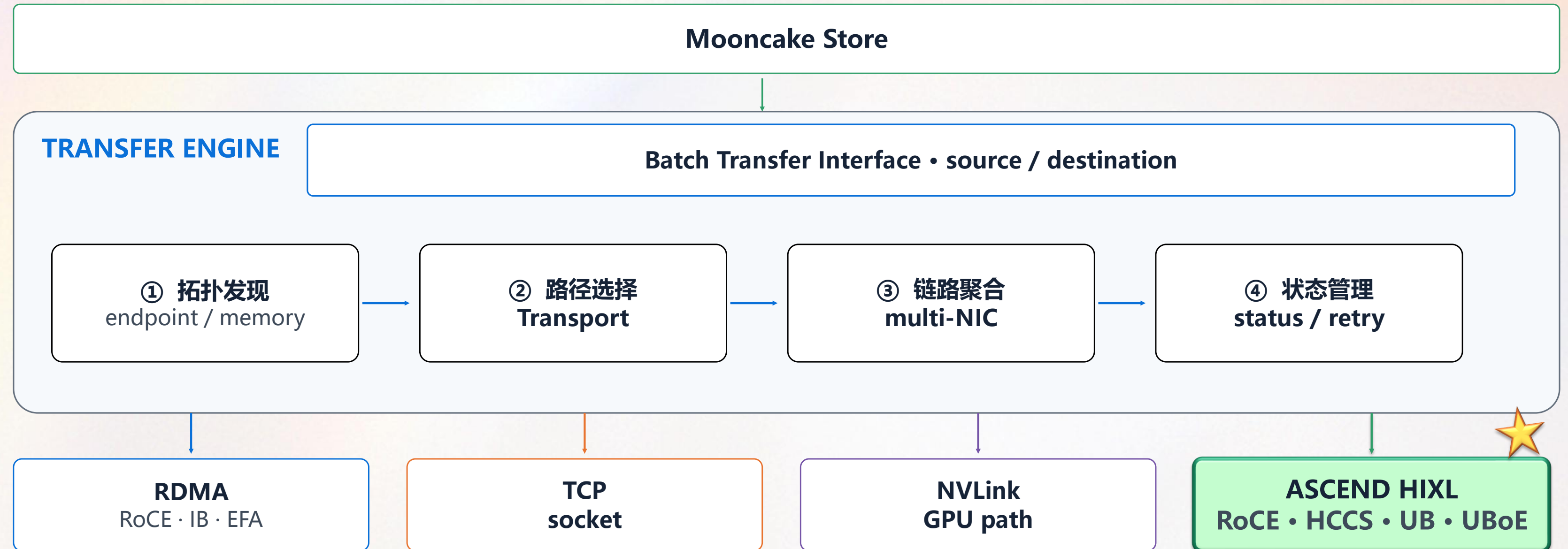
KV Cache占用内存空间随序列长度线性增长, 需要跨机的多级KV Pool

Mooncake Store多级KV Pool



L0 存最热的系统KV, L1 存用户最近请求的热KV, L2 存用户长时间未重复使用的冷KV, Mooncake Store 把外部存储提供成统一的KV Pool

Transfer Engine: 统一接口与多种传输后端



Store使用Transfer Engine进行传输, **HIXL**在其中提供了Ascend传输后端

HIXL 分层架构: Engine、CS、HCOMM 与底层协议

HIXL ENGINE Initialize · RegisterMem · Connect · TransferSync/Async · GetTransferStatus · Disconnect

HIXL CS

SERVER ROLE
RegMem · Listen

CLIENT ROLE
Connect · BatchPut / BatchGet · QueryCompleteStatus

HCOMM Endpoint · Memory Register / Export / Import · Channel

RoCE

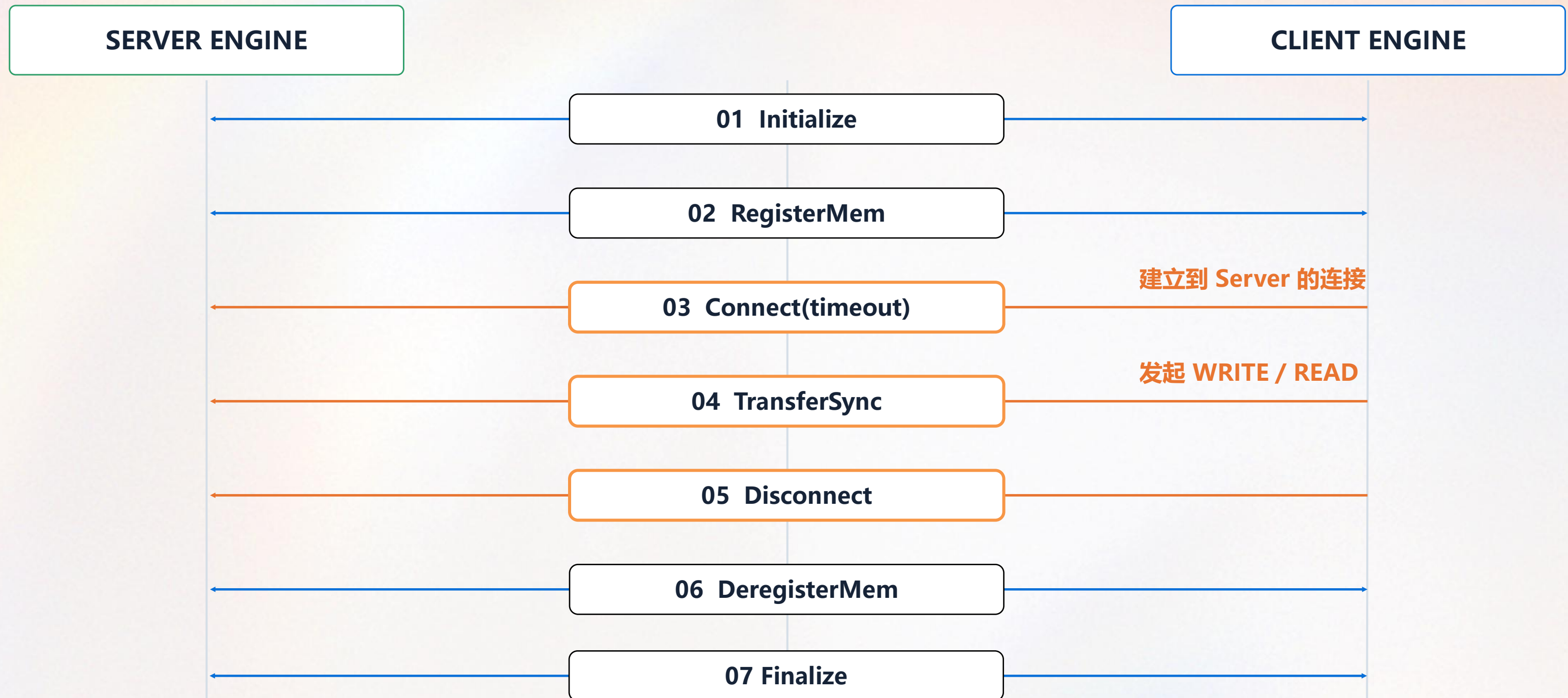
HCCS

UB

UBoE

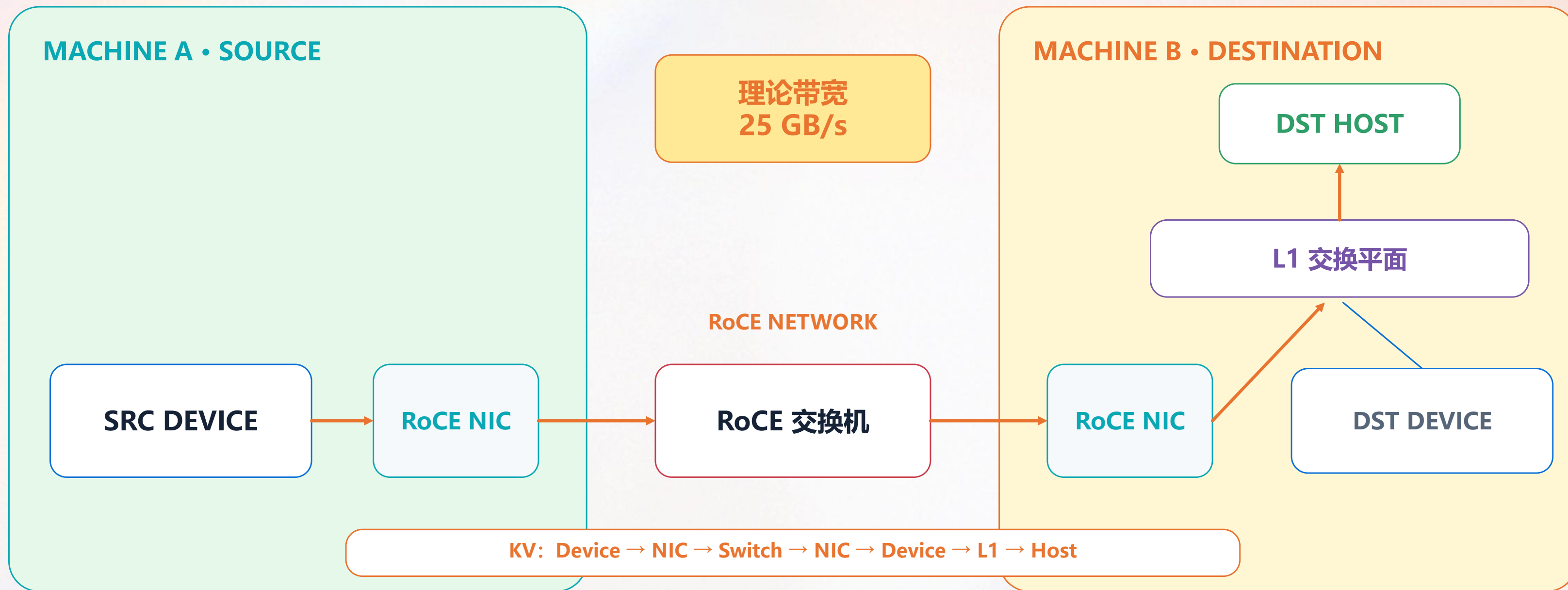
HIXL对上提供极简API, 对下屏蔽底层通信协议差异

HIXL 接口调用示例



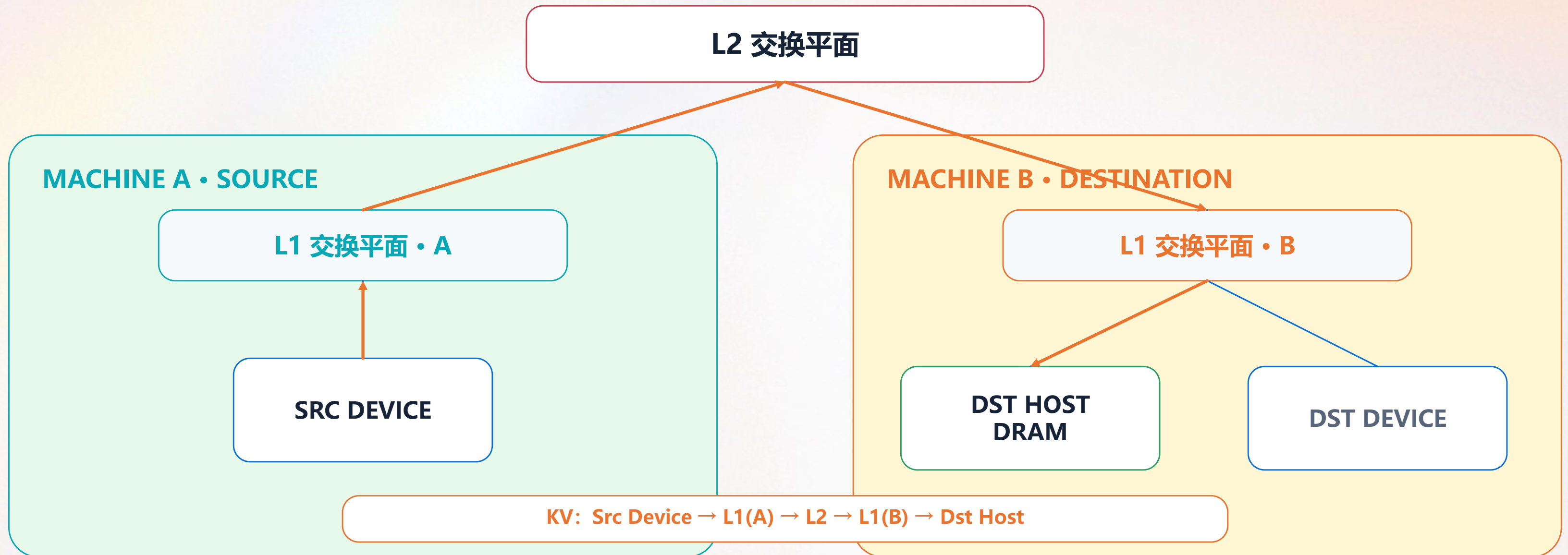
整个单边通信生命周期只需要调用7个接口

KV Pool底层传输方式一: RoCE



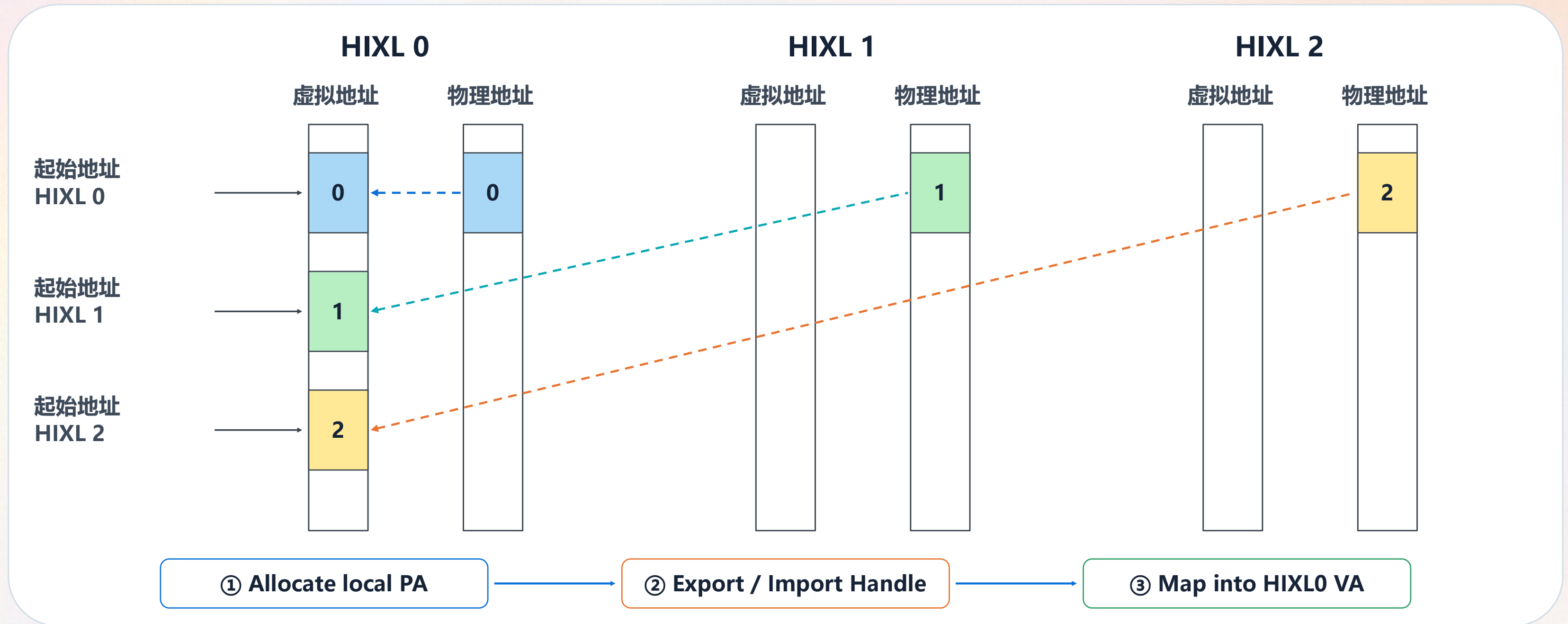
RoCE: Src Device → 网卡 → 交换机 → Dst 网卡 → Dst Device → L1 交换平面 → Host

KV Pool底层传输方式二: HCCS-FabricMem



HCCS-FabricMem: Src Device → Src L1 → L2 交换平面 → Dst L1 → Dst Host

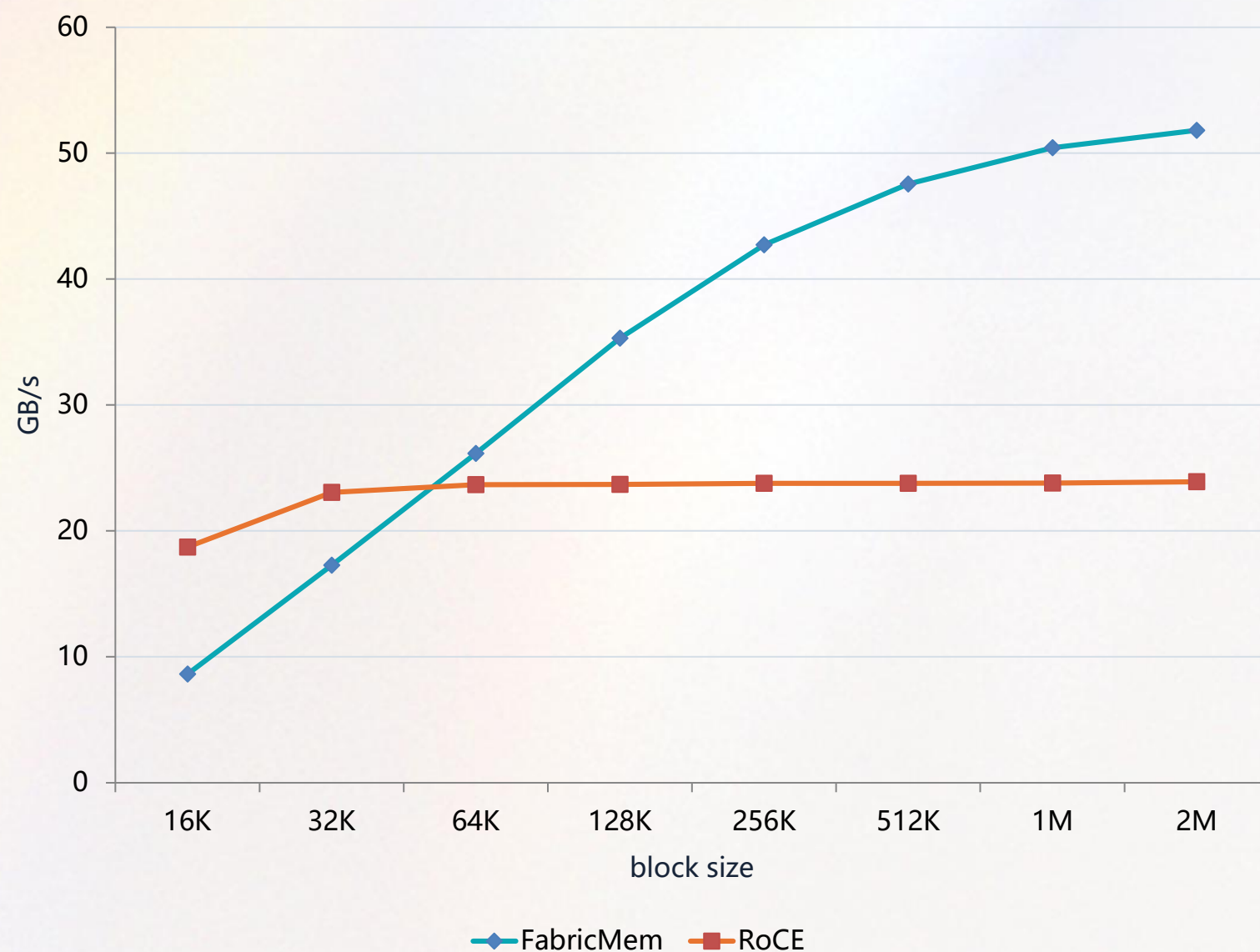
KV Pool底层传输方式二: HCCS-FabricMem



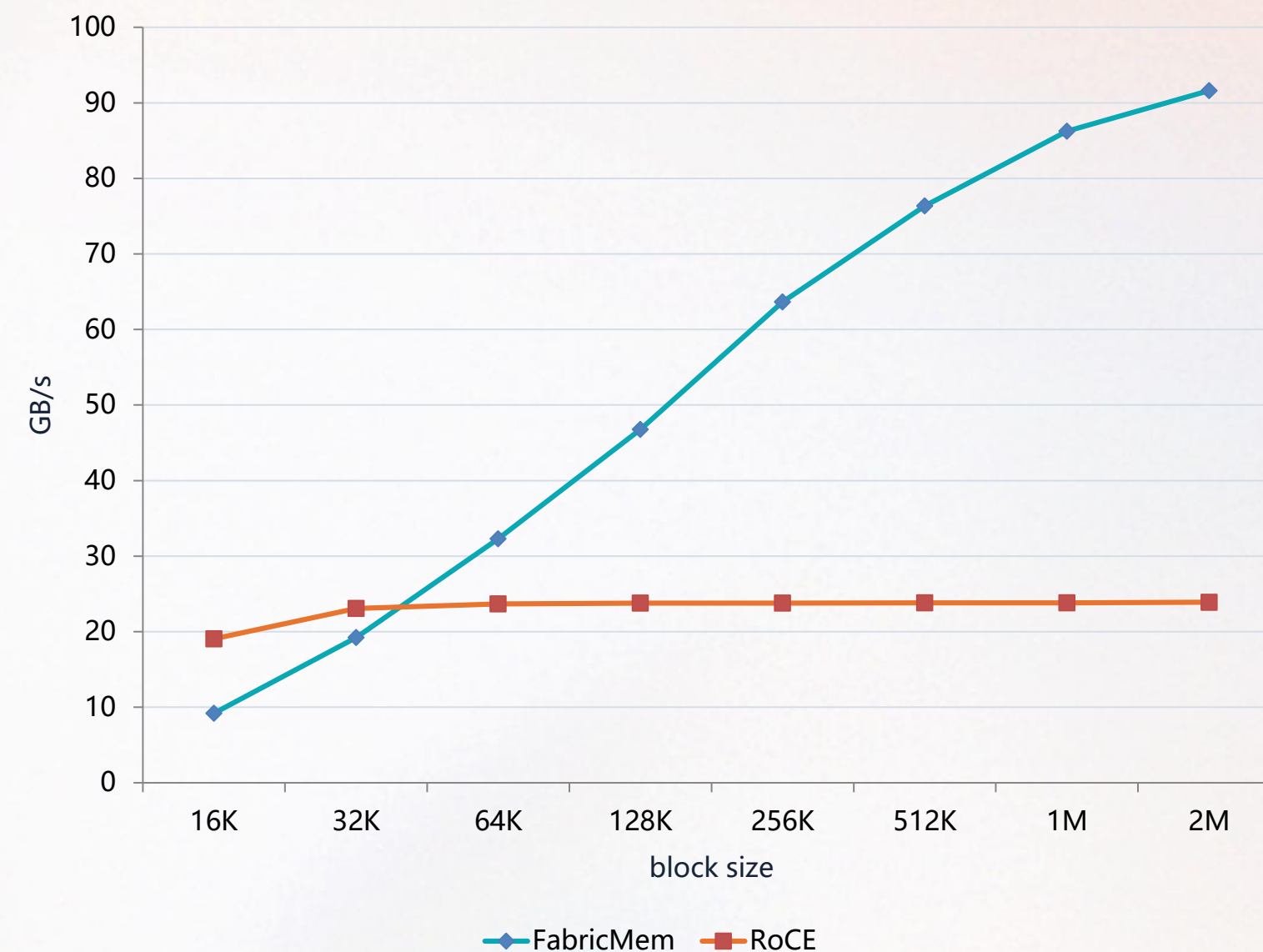
本地进程申请大块虚拟地址, 利用统一编址的特性将远端的物理地址映射到本地的虚拟地址上

FabricMem 与 RoCE 的 D2rH / rH2D 带宽对比

D2rH • Device → remote Host



rH2D • remote Host → Device



在KVPool典型传输方向上, FabricMem在超过64K的数据时带宽远超RoCE, 在KV Pool中能获得更多收益

推理部署流程

• 01 • 启动mooncake_master

```
mooncake_master \  
  --port 50088 \  
  --eviction_high_watermark_ratio 0.9 \  
  --eviction_ratio 0.1
```

• 02 • 配置客户端option

```
{  
  "protocol": "ascend",  
  "global_segment_size": "128GB"  
  "metadata_server": "P2PHANDSHAKE",  
  "master_server_address": "<master_ip>:50088",  
}  
export MOONCAKE_CONFIG_PATH=/etc/mooncake/mooncake.json
```

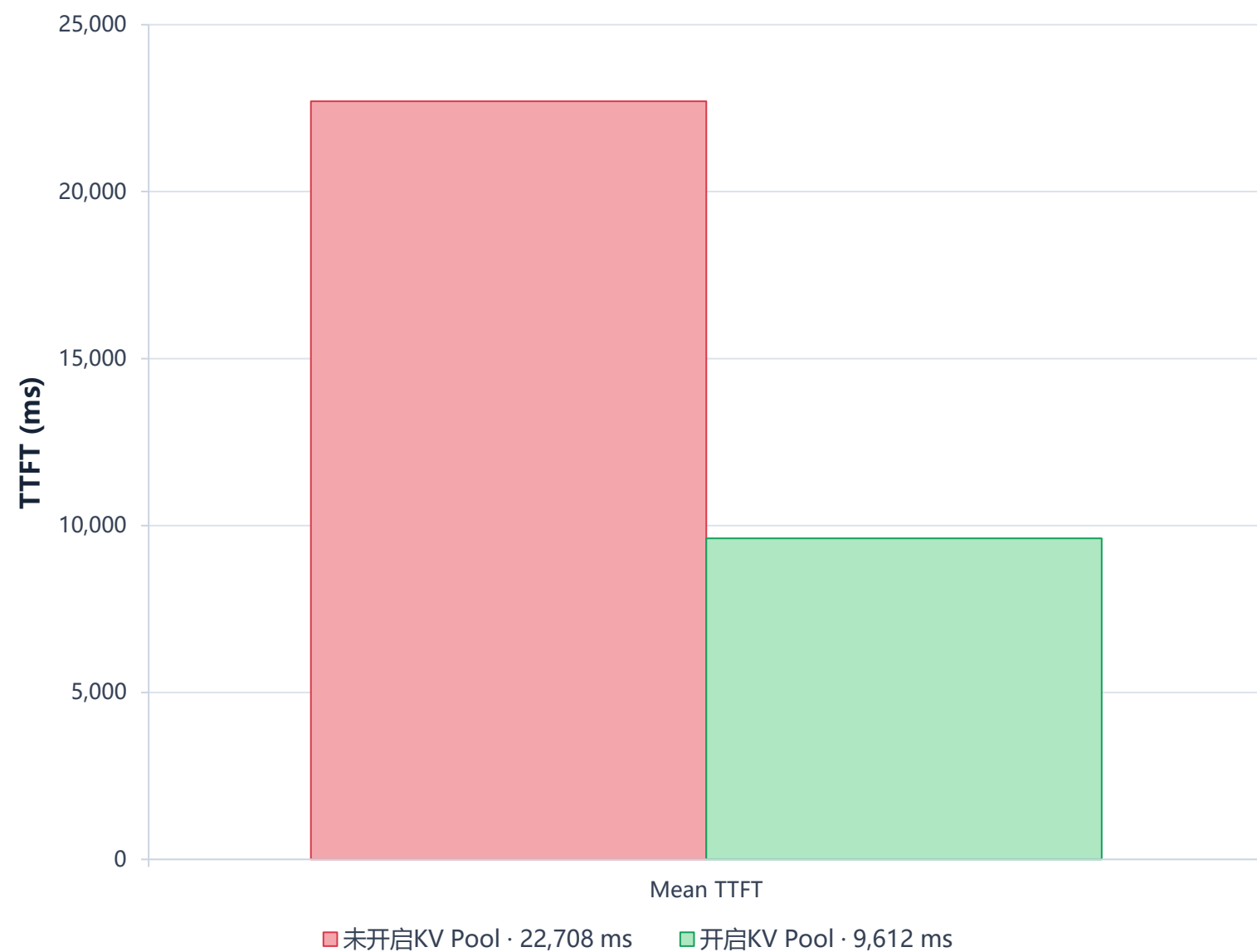
• 03 • 确定传输方式, 启动服务

```
# 使用FabricMem进行KVPool传输  
export ASCEND_ENABLE_USE_FABRIC_MEM=1  
  
vllm serve --kv-transfer-config '{  
  "kv_connector": "AscendStoreConnector",  
  "kv_role": "kv_producer",  
  "kv_connector_extra_config": {  
    "backend": "mooncake",  
    "lookup_rpc_port": "21001"  
  }  
'
```

先启动mooncake_master, 再配置客户端, 最后确定传输方式, 启动服务

线上服务TTFT 降低 57.7%

TTFT Mean · 线上 2k 条数据: 平均300K上下文



归一化 TTFT · 使用KV Pool下载57.7%



16并发 · 4DP/8TP/32EP · DeepSeek-V4-Flash · 命中率85%: 22,708ms → 9,612ms (-57.7%)

未来演进规划

01 Layer-wise 传输

按层流水，计算与传输重叠

02 基于KV Pool的KV Offload

解决超长上下文场景问题

03 下一代基于 UB-CTP / UBoE / UB-RTP / UBMEM 等协议的 KV Pool

面向下一代高速互联

04 更丰富的多层缓存架构

扩展缓存层级与统一调度



THANKS



参与互动答疑贴赢取礼品

